

Tree Traversals

Tree traversals are systematic ways to visit all nodes in a tree exactly once. The main traversals are inorder, preorder, and postorder.

Inorder Traversal

Order: left subtree, current node, right subtree.

Pseudocode:

```
INORDER(node)
    if node is null:
        return
    INORDER(node.left)
    process(node)
    INORDER(node.right)
```

Special property: on a [[Binary Search Trees|binary search tree]], inorder visits nodes in sorted order (ascending).

Time: $O(n)$ (visit each node once)

Preorder Traversal

Order: current node, left subtree, right subtree.

Pseudocode:

```
PREORDER(node)
    if node is null:
        return
    process(node)
    PREORDER(node.left)
    PREORDER(node.right)
```

Use: copying a tree structure, printing with proper nesting visible.

Time: $O(n)$

Postorder Traversal

Order: left subtree, right subtree, current node.

Pseudocode:

```

POSTORDER(node)
    if node is null:
        return
    POSTORDER(node.left)
    POSTORDER(node.right)
    process(node)

```

Use: deleting a tree (children must be deleted before parent), computing tree properties that depend on child values (e.g., tree height, subtree sums).

Time: $O(n)$

Level-Order Traversal (BFS Style)

Order: visit nodes by depth (breadth-first).

Implementation: use a [[Queues|queue]].

```

LEVELORDER(root)
    queue = empty queue
    queue.ENQUEUE(root)
    while not queue.ISEMPTY():
        node = queue.DEQUEUE()
        process(node)
        if node.left is not null:
            queue.ENQUEUE(node.left)
        if node.right is not null:
            queue.ENQUEUE(node.right)

```

Time: $O(n)$

Comparison

Traversal	Order	Use Case
Inorder	left, node, right	sorted output (BST); symmetric view
Preorder	node, left, right	copy; printing hierarchy
Postorder	left, right, node	deletion; computing subtree properties
Level-order	by depth	breadth-first; layer-by-layer

Iterative vs Recursive

- **Recursive:** natural fit for tree structure; uses call stack implicitly
 - **Iterative:** explicit `[[Stacks|stack]]` for inorder/preorder/postorder, or `[[Queues|queue]]` for level-order; useful if recursion depth is problematic
-

Related

- `[[Trees]]`
- `[[Binary Search Trees]]`
- `[[Stacks]]`
- `[[Queues]]`
- `[[Depth-First Search]]`
- `[[Breadth-First Search]]`